



**NATIONAL RESEARCH UNIVERSITY OF INFORMATION
TECHNOLOGY, MECHANICS AND OPTICS**

FACULTY OF CONTROL SYSTEMS AND ROBOTICS

**Smooth Trajectory Planning for ABB IRB 140 Robot via
State-Space Interpolation and Simulation in CoppeliaSim: A
Comparative Study of Optimization Algorithms**

COMPLETED BY: Valverde Ramírez, Edgar David

ISU: 520193

GROUP: R4136c

DISCIPLINE: Simulation of Robotic Systems

INSTRUCTOR: PhD. Ivan Borisov

ASSISTANT: Rakshin, Egor

Contenido

1. INTRODUCTION:	3
1.1 Background and Context:	3
1.2 Problem Statement	4
1.3 Motivation	5
1.4 Project Objectives	6
2. ROBOT MODELING AND KINEMATICS:	7
2.1 The ABB IRB 140 Manipulator	7
2.2 Denavit-Hartenberg (DH) Parameters	9
2.3 Forward Kinematics (FK) Implementation	9
3. MATHEMATICAL FRAMEWORK:	11
3.1 State-Space Kinematic Modeling	11
3.2 Prediction Matrices	13
3.3 Cauchy Formula for Repeated Integration	14
4. OPTIMIZATION STRATEGIES:	15
4.1 The Cost Function	15
4.2 Gradient-Based Optimization: SLSQP	16
4.3 Evolutionary Optimization: Genetic Algorithm (GA)	17
4.4 Comparative Hypothesis	19
5. SOFTWARE IMPLEMENTATION & SIMULATION SETUP	19
5.1 System Architecture	19
5.2 Python Optimization Modules	20
5.3 CoppeliaSim Integration (player.py)	21
5.4 Experimental Configuration	22
6. RESULTS AND COMPARATIVE ANALYSIS	23
6.1 Computational Efficiency and Convergence	23
6.2 Trajectory Profile Analysis	26
6.3 Convergence Analysis	28
6.4 Discussion	28
6.5 Simulation Validation	29
7. CONCLUSIONS	30

1. INTRODUCTION:

1.1 Background and Context:

The field of industrial automation has evolved significantly over the last few decades, transitioning from rigid, hard-coded sequences to flexible, intelligent systems capable of complex manipulation tasks. In this context, the **ABB IRB 140**, a compact 6-degree-of-freedom (6-DOF) industrial manipulator, represents a standard class of robots used for assembly, arc welding, and material handling. For such robots to operate efficiently and safely, **Trajectory Planning** is a fundamental requirement.

Trajectory planning involves generating a time-based sequence of values for the robot's joints ($q_1, \dots, q_6$) to move the end-effector from a starting pose to a target pose. Traditionally, this process is decoupled into two stages:

1. **Path Planning:** Determining the geometric path in the workspace (often using Inverse Kinematics to find discrete waypoints).
2. **Trajectory Generation:** Interpolating these waypoints using polynomials (e.g., Cubic or Quintic splines) to ensure continuity in velocity and acceleration.

However, modern control strategies seek to integrate these stages to optimize performance metrics such as energy consumption, time, or motion smoothness. Smoothness is particularly critical; discontinuities in acceleration (infinite jerk) cause excessive vibrations, wear on mechanical gears, and tracking errors in the control loops.

This project explores a **State-Space Interpolation** approach, as proposed by *Orbegoso, Valverde et al.*, which treats trajectory generation as a dynamic system where the control signal is a high-order derivative of the position (such as Jerk or Jounce). By integrating this control signal, mathematical continuity (C^2 or C^3) is inherently guaranteed.

Furthermore, this project expands upon the baseline methodology by introducing a **comparative study of optimization algorithms**, specifically contrasting gradient-based methods (SLSQP) against evolutionary strategies (Genetic Algorithms).

1.2 Problem Statement

The central problem addressed in this project is the generation of smooth, collision-free, and kinematically feasible trajectories for the ABB IRB 140 robot without relying on pre-calculated Inverse Kinematics (IK) solutions for every point in the path.

Standard trajectory planning faces several distinct challenges:

1. **Kinematic Constraints:** The robot has strict physical limits on joint range (q_{min}, q_{max}) and joint velocity ($\dot{q}_{max}$). Violating these limits triggers emergency stops or damages the hardware. Traditional polynomial interpolation often requires iterative "trial-and-error" tuning to satisfy these bounds.
2. **Non-Linearity:** The Forward Kinematics (FK) of a 6-DOF articulated arm involves complex trigonometric transformations. Mapping a desired Cartesian change (in $x, y, z, Roll, Pitch, Yaw$) to joint space is highly non-linear.
3. **Local vs. Global Minima:** When optimizing a trajectory to reach a target, the objective function (minimizing error) is often non-convex.
 - **Gradient-based solvers** (like SLSQP) are efficient but may get trapped in local minima, failing to find a solution if the initial guess is poor.
 - **Stochastic solvers** (like Genetic Algorithms) are better at exploring the search space but are computationally expensive and may produce "jittery" solutions that are suboptimal for smooth control.

Therefore, the problem is not merely to move the robot, but to find an optimal control sequence U in the state space that minimizes the final pose error while strictly respecting all kinematic constraints, and to determine which algorithmic approach offers the best balance between accuracy, smoothness, and convergence speed.

1.3 Motivation

The motivation for this project is twofold: **methodological advancement** and **algorithmic benchmarking**.

From a **methodological** perspective, implementing the State-Space approach allows for "direct" trajectory planning. Instead of solving the Inverse Kinematics problem analytically (which can be difficult for redundant robots or singular configurations), this method solves it numerically as an optimization problem over a time horizon. This allows for the integration of higher-order smoothness constraints (Jerk) directly into the motion model, ensuring that the resulting trajectory is safe for high-speed operation.

From an **algorithmic** perspective, there is often a trade-off in robotics between speed and robustness.

- **SciPy's SLSQP (Sequential Least Squares Programming)** represents the industry standard for non-linear optimization with constraints. It relies on the Jacobian and Hessian of the objective function. It is expected to be fast but sensitive to initialization.
- **DEAP (Distributed Evolutionary Algorithms in Python)** represents the metaheuristic approach. Genetic Algorithms (GA) mimic natural selection, maintaining a population of solutions. This is motivated by the need to avoid local minima in complex manipulator postures where a gradient descent might fail.

By implementing and comparing these two distinct paradigms within a unified simulation environment (**CoppeliaSim**), this project provides

empirical data on which strategy is better suited for offline trajectory planning of industrial manipulators.

1.4 Project Objectives

The primary goal of this work is to design, implement, and validate a smooth trajectory planner for the ABB IRB 140 robot using state-space interpolation. This is achieved through the following specific objectives:

1. Kinematic Modeling:

- Derive the Denavit-Hartenberg (DH) parameters for the ABB IRB 140.
- Implement the Forward Kinematics (FK) engine in Python to map joint states to end-effector poses.

2. Algorithm Development:

- Formulate the motion model in State-Space: $X_{i+1} = AX_i + BU_i$.
- Implement the **Cauchy formula for repeated integration** to reconstruct joint positions from the control signal (Jerk/Jounce).

3. Optimization Implementation:

- **Strategy A:** Implement a Gradient-based solver using `scipy.optimize.minimize` (Method: SLSQP) with defined bounds for position and velocity.
- **Strategy B:** Implement an Evolutionary solver using the DEAP library (Genetic Algorithm) with appropriate crossover, mutation, and selection operators.

4. Simulation and Validation:

- Establish a communication bridge between Python and **CoppeliaSim** (using the ZeroMQ Remote API).
- Visualise the generated trajectories on the virtual ABB IRB 140.

- Conduct a comparative analysis of both optimization strategies based on **Final Positioning Error (mm)**, **Execution Time (s)**, and **smoothness profiles**.

5. Performance Analysis:

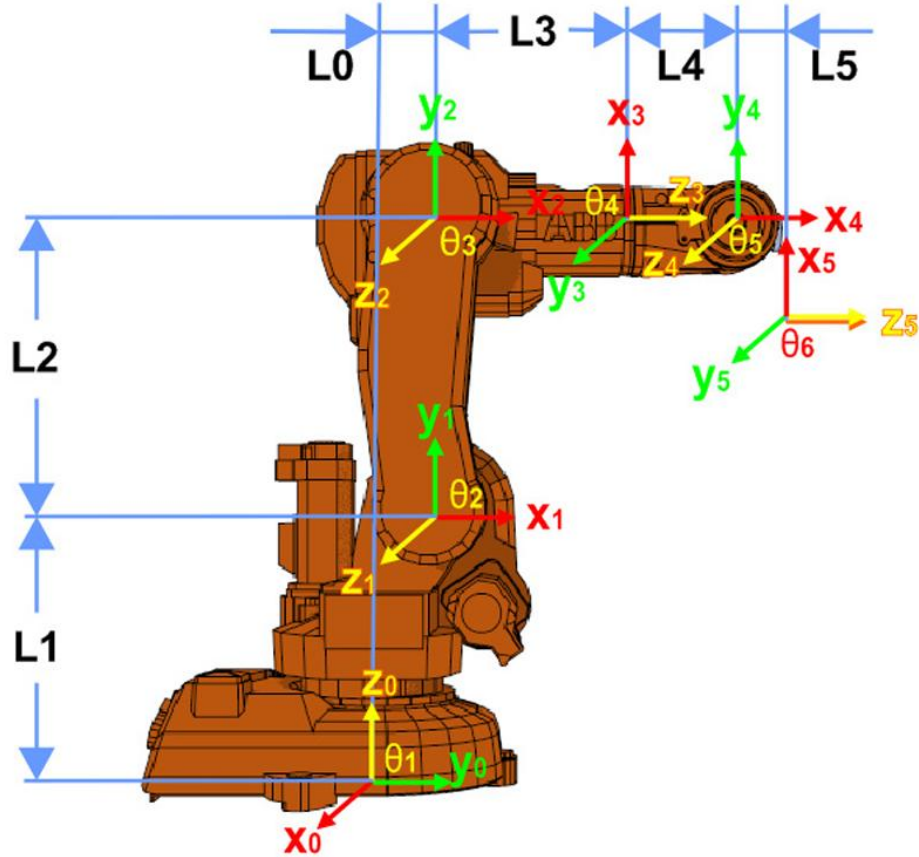
- Demonstrate that the generated trajectories respect the hardware limits: $\approx 150^\circ/\text{s}$ velocity and specific joint ranges (e.g., Axis 1: $\pm 180^\circ$).

2. ROBOT MODELING AND KINEMATICS:

2.1 The ABB IRB 140 Manipulator

The robotic platform selected for this study is the **ABB IRB 140**. This is a compact, six-axis, multipurpose industrial robot widely used in industries for arc welding, assembly, and material handling applications. It is characterized by its high acceleration, large working envelope relative to its base size, and high payload capacity for its class (6 kg). For the purpose of trajectory planning and simulation, the robot is modeled as a series of rigid links connected by revolute joints. The mechanical structure corresponds to a standard anthropomorphic arm with a spherical wrist.

Figure 1 ABBIRB-140robot



2.1.1 Mechanical Specifications

The key physical parameters that define the workspace and the operational limits of the robot are presented below. These constraints are critical for the optimization algorithms (SLSQP and Genetic Algorithms) to ensure the generated solutions are feasible.

Table 2.1: *Kinematic Limits of the ABB IRB 140*

Axis	Type	Motion Range (Degrees)	Motion Range (Radians)	Max Velocity (°/s)
Joint 1	Rotation	+180° to -180°	± 3.14	200
Joint 2	Arm	+110° to -100°	3.82 to -1.74	200
Joint 3	Arm	+50° to -230°	1.74 to -4.01	260
Joint 4	Wrist	+200° to -200°	± 3.49	360
Joint 5	Bend	+115° to -115°	± 2.00	360
Joint 6	Turn	+400° to -400°	± 6.98	450

2.2 Denavit-Hartenberg (DH) Parameters

To mathematically represent the robot's geometry, the **Denavit-Hartenberg (DH)** convention is employed. This method describes the relative position and orientation of coordinate frames attached to each link of the robot.

The position and orientation of frame i relative to frame $i - 1$ are determined by four parameters:

1. **Link Length (a_i):** Distance along x_i from the intersection of the x_i and z_{i-1} axes to the origin of frame i .
2. **Link Twist (α_i):** Angle between z_{i-1} and z_i measured about x_i .
3. **Link Offset (d_i):** Distance along z_{i-1} from the origin of frame $i - 1$ to the intersection of the x_i and z_{i-1} axes.
4. **Joint Angle (θ_i):** Angle between x_{i-1} and x_i measured about z_{i-1} .

For the ABB IRB 140, the standard DH parameters are derived based on the frame assignment where the z-axis aligns with the axis of rotation for each joint.

Table 2.2: *Standard DH Parameters for ABB IRB 140*

Link (i)	θ_i (Variable)	d_i (mm)	a_i (mm)	α_i (rad)
1	$q_1 + \pi/2$	352	70	$\pi/2$
2	$q_2 + \pi/2$	0	360	0
3	q_3	0	0	$\pi/2$
4	q_4	380	0	$-\pi/2$
5	q_5	0	0	$\pi/2$
6	q_6	65	0	0

2.3 Forward Kinematics (FK) Implementation

Forward Kinematics (FK) is the process of calculating the position and orientation of the end-effector (Frame 6) given the set of joint angles $Q = [q_1, q_2, q_3, q_4, q_5, q_6]^T$. This is a crucial component of the project's optimization loop, as the "Fitness Function" or "Cost Function" requires comparing the *calculated* end-effector position against the *target* position.

2.3.1 Transformation Matrices

Using the DH parameters defined in Table 2.2, the homogeneous transformation matrix ${}^{i-1}T_i$ transforming a vector from Frame i to Frame $i - 1$ is given by:

$${}^{i-1}T_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.3.2 Chain Calculation

The total transformation from the Base Frame (0) to the End-Effector Frame (6) is obtained by multiplying the individual link matrices sequentially:

$${}^0T_6 = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6$$

The resulting matrix

$0T_6$

has the form:

$${}^0T_6 = \begin{bmatrix} r_{11} & r_{12} & r_{13} & p_x \\ r_{21} & r_{22} & r_{23} & p_y \\ r_{31} & r_{32} & r_{33} & p_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where:

- $[p_x, p_y, p_z]^T$ represents the Cartesian position of the tool center point (TCP).
- The 3×3 submatrix $R = [r_{ij}]$ represents the orientation (Rotation Matrix).

2.3.3 Software Implementation (Python)

In this project, the Forward Kinematics engine was implemented in Python using the NumPy library to ensure high-performance matrix multiplication. This function, `forward_kinematics(q)`, serves as the primary evaluation step in both the SLSQP and Genetic Algorithm loops.

```
def _dh_matrix(self, theta, d, a, alpha):
    """Standard DH Transformation Matrix."""
    c_th, s_th = np.cos(theta), np.sin(theta)
    c_alp, s_alp = np.cos(alpha), np.sin(alpha)

    # Standard DH Matrix
    return np.array([
        [c_th, -s_th*c_alp, s_th*s_alp, a*c_th],
        [s_th, c_th*c_alp, -c_th*s_alp, a*s_th],
        [0, s_alp, c_alp, d],
        [0, 0, 0, 1]
    ])
```

This implementation was validated against the internal kinematic solver of CoppeliaSim to ensure that the error was negligible ($<10^{-5}$ m) before proceeding to the trajectory optimization phase.

3. MATHEMATICAL FRAMEWORK:

The fundamental point of this project lies in moving away from traditional polynomial splines (which solve boundary conditions analytically) to a **State-Space Model**. In this model, the trajectory is generated by a dynamic system where the input control signal is a higher-order derivative of the position. This ensures that the resulting motion is mathematically continuous and differentiable by design.

3.1 State-Space Kinematic Modeling

Standard kinematics assumes that the robot moves between points, but typically requires inverse kinematics at every step or fits a polynomial curve after the fact. Instead, we model the evolution of the joint angles as a discrete linear system.

3.1.1 System Definition

Let $q(t)$ be the position of a single joint. If we wish to control the n -th derivative of the position (for example, if $n = 3$, we control the Jerk), we can define the state vector $x(t)$ at time instance i as containing the position and its derivatives up to order $n - 1$:

$$\vec{x}_{(i)} = \begin{bmatrix} q_{(i)} \\ \dot{q}_{(i)} \\ \ddot{q}_{(i)} \\ \vdots \\ q_{(i)}^{(n-1)} \end{bmatrix}$$

The control input $u_{(i)}$ is defined as the n -th derivative, which is held constant over the discretization time interval Δt :

$$u_{(i)} = q_{(i)}^{(n)}$$

Using a Taylor series expansion (or simple integration over Δt), the state at the next time step $(i+1)$ can be predicted from the current state and the control input. This yields the discrete state-space equation:

$$\vec{x}_{(i+1)} = A\vec{x}_{(i)} + Bu_{(i)}$$

Where the system matrix A (describing the free evolution of the system) and the input matrix B (describing the effect of the control) are given by:

$$A = \begin{bmatrix} 1 & \Delta t & \frac{\Delta t^2}{2!} & \cdots & \frac{\Delta t^{n-1}}{(n-1)!} \\ 0 & 1 & \Delta t & \cdots & \frac{\Delta t^{n-2}}{(n-2)!} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}_{[n \times n]} \quad B = \begin{bmatrix} \frac{\Delta t^n}{n!} \\ \frac{\Delta t^{n-1}}{(n-1)!} \\ \vdots \\ \Delta t \end{bmatrix}_{[n \times 1]}$$

3.1.2 Multi-Joint Extension

The ABB IRB 140 has $m = 6$ joints. Therefore, the global system state X is formed by stacking the state vectors of all individual joints. The matrices are expanded into block-diagonal forms:

$$X_{(i+1)} = \mathbb{A}X_{(i)} + \mathbb{B}U_{(i)}$$

Where:

- $X_{(i)} \in \mathbb{R}^{n \cdot m}$ is the global state vector.
- $U_{(i)} \in \mathbb{R}^m$ is the control vector containing the n -th derivative for all 6 joints.
- $\mathbb{A}$ is a block-diagonal matrix containing m copies of A .
- $\mathbb{B}$ is a block-diagonal matrix containing m copies of B .

3.2 Prediction Matrices

To optimize the trajectory, we need to predict the final state of the robot after a sequence of k time steps (where k is the prediction horizon or the total number of steps in the movement). Instead of iterating the state equation k times inside a loop (which is computationally slow for gradients), we can formulate a vectorized prediction model.

By recursively substituting the state equation, the state at step k can be expressed as a function of the initial state $X_{(0)}$ and the sequence of control inputs $U = [U_{(0)}, U_{(1)}, \dots, U_{(k-1)}]^T$.

The prediction equation is:

$$X_{(k)} = \mathcal{M}X_{(0)} + \mathfrak{N}U$$

Here, the matrices are defined as follows:

1. The Free Response Matrix ($\mathcal{M}$):

This matrix projects the initial state forward in time, assuming zero control input. It is simply the power of the system matrix:

$$\mathcal{M} = \mathbb{A}^k$$

3. The Forced Response Matrix ($\mathfrak{N}$):

This matrix (often called the Convolution Matrix in MPC literature) describes how each control input in the sequence affects the final state. Since earlier inputs have more time to integrate, they have higher powers of $\mathbb{A}$ attached to them:

$$\mathfrak{N} = [\mathbb{A}^{k-1}\mathbb{B} \quad \mathbb{A}^{k-2}\mathbb{B} \quad \dots \quad \mathbb{A}\mathbb{B} \quad \mathbb{B}]$$

Significance:

Equation $X_{(k)} = \mathcal{M}X_{(0)} + \mathbf{R}\mathbf{U}$ is critical for the optimization in Chapter 4. It allows us to express the final joint positions $q_{(k)}$ as a linear function of the unknown variables $\mathbf{U}$. This linearity simplifies the gradient calculation for the optimizer.

3.3 Cauchy Formula for Repeated Integration

Once the optimization algorithms (SLSQP or GA) determine the optimal control vector $\mathbf{U}$ (which represents, for example, the Jerk profile), we must reconstruct the actual position trajectory $q(t)$ to send to the robot simulator.

Since the control signal $U(t)$ is modeled as piecewise constant (step functions) over the intervals, the position is obtained by integrating this signal n times.

The **Cauchy formula for repeated integration** states that the n -th integral of a function $f(t)$ can be compressed into a single integral using a kernel $(t-w)^{n-1}$:

$$q(t) = \int_{t_0}^t \frac{(t-w)^{n-1}}{(n-1)!} U(w) dw + \sum_{j=0}^{n-1} \frac{q^{(j)}(t_0)}{j!} (t - t_0)^j$$

In the context of our discrete simulation:

1. The control signal U essentially defines the **Jerk** (if $n = 3$).
2. Integrating Jerk once yields **Acceleration** (continuous piecewise linear).
3. Integrating Acceleration yields **Velocity** (parabolic smooth).
4. Integrating Velocity yields **Position** (cubic smooth).

This mathematical guarantee ensures that the resulting trajectory is C^{n-1} continuous. For the ABB IRB 140, this smoothness is essential to prevent high-frequency oscillations in the torque commands, thereby reducing mechanical stress and improving tracking accuracy in the simulator.

4. OPTIMIZATION STRATEGIES

With the state-space model defined, the trajectory planning problem translates into finding the optimal sequence of control inputs $\mathbf{U} = [U_0, U_1, \dots, U_{k-1}]$ that moves the robot from an initial state X_{start} to a state that satisfies the target kinematic conditions, while respecting physical limits.

This chapter details the cost function design and the two distinct optimization algorithms implemented to solve it: **Sequential Least Squares Programming (SLSQP)** and **Genetic Algorithms (GA)**.

4.1 The Cost Function

The objective of the optimization is to minimize the error between the robot's final state and the desired target state in the Cartesian Task Space.

The total Loss Function $L(\mathbf{U})$ is composed of two primary terms: an accuracy term and a regularization term.

$$L(\mathbf{U}) = \underbrace{\| Y_{target} - FK(X_{(k)}) \|^2}_{\text{Accuracy Cost}} + \underbrace{\lambda \sum_{i=0}^{k-1} \| U_i \|^2}_{\text{Smoothness/Energy Cost}}$$

4.1.1 Accuracy Cost

This term measures the Euclidean distance between the desired end-effector pose and the actual pose achieved at step k .

- $X_{(k)}$ is the predicted final joint state derived from equation: $X_{(k)} = \mathcal{M}X_{(0)} + \mathbf{N}\mathbf{U}$.
- $FK(X_{(k)})$ applies the Forward Kinematics (from Chapter 2) to the position components of $X_{(k)}$.
- Y_{target} includes the target $[x, y, z]$ coordinates and potentially orientation (Quaternion or Euler angles).

4.1.2 Regularization Term (λ)

In trajectory planning, "greedy" optimization can lead to erratic behavior, where the robot accelerates violently to reach the target 1 millisecond faster. To prevent this, we add a penalty on the magnitude of the control input U (e.g., Jerk).

- λ is a weighting hyperparameter.
- A higher λ forces the trajectory to be smoother but might result in a slightly larger final position error.
- A lower λ allows for aggressive movements to minimize position error perfectly.

4.2 Gradient-Based Optimization: SLSQP

The first strategy employs **Sequential Least Squares Programming (SLSQP)**, provided by the `scipy.optimize` library. This is a local optimization method that uses the derivatives (gradients) of the cost function to find the minimum.

4.2.1 Algorithm Overview

SLSQP is ideal for problems with:

1. Smooth, continuous objective functions.
2. Explicit boundaries (inequality constraints).
3. Equality constraints.

In Python implementation, the solver is configured as follows:

- **Decision Variables:** The flattened vector $\mathbf{U}$ of size $k \times m$ (horizon $\times$ joints).
- **Initial Guess (u_0):** A vector of zeros, assuming the robot starts with no Jerk.
- **Bounds:** Each element of $\mathbf{U}$ is constrained to valid physical limits to prevent impossible acceleration spikes.

4.2.2 Handling Constraints

Crucially, the robot has strict limits on Joint Position (q) and Joint Velocity ($\dot{q}$). Since $X_{(k)}$ depends linearly on $\mathbf{U}$, these constraints can be formulated as linear inequalities:

$$q_{min} \leq \mathcal{M}_q X_{(0)} + \mathfrak{N}_q \mathbf{U} \leq q_{max}$$

$$\dot{q}_{min} \leq \mathcal{M}_{\dot{q}} X_{(0)} + \mathfrak{N}_{\dot{q}} \mathbf{U} \leq \dot{q}_{max}$$

These are passed to `minimize()` as a constraints dictionary or via the `bounds` parameter, ensuring that the solver never explores "illegal" regions of the state space.

4.3 Evolutionary Optimization: Genetic Algorithm (GA)

The second strategy uses a **Genetic Algorithm (GA)**, implemented via the DEAP library. Unlike SLSQP, GA does not require gradients. It mimics biological evolution to explore the search space globally, making it potentially more robust to local minima but computationally more expensive.

4.3.1 Chromosome Representation

An "individual" in the population represents a candidate trajectory.

- **Genotype:** A list of floating-point numbers representing the entire control sequence $\mathbf{U}$.
- **Length:** $k \times m$ (same as SLSQP).

4.3.2 The Evolutionary Cycle

The algorithm proceeds through generations:

1. **Initialization:** A population of N random control sequences is generated.
2. **Evaluation (Fitness):** Each individual is decoded, passed through the Prediction Matrix (Eq 3.5), and evaluated using the Cost Function $L(\mathbf{U})$.
 - *Note:* Since GAs typically maximize fitness, we minimize the cost by defining $\text{Fitness} = 1/(L(\mathbf{U}) + \epsilon)$.
 - *Penalty Handling:* If a trajectory violates joint/velocity limits, a heavy penalty is added to the cost, effectively "killing" that individual in the selection process.

3. **Selection:** We use **Tournament Selection** (size=3). The best individuals are chosen to become parents.
4. **Crossover (Recombination):** We use **Two-Point Crossover**. Two parents swap segments of their control sequence to create offspring, combining "good" parts of different trajectories.
5. **Mutation:** We use **Gaussian Mutation**. Random noise is added to the control values with a probability P_{mut} . This maintains diversity and prevents the population from stagnating.

4.3.3 Implementation with DEAP

The Python implementation leverages the DEAP framework:

```
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("attr_float", np.random.uniform, u_bounds[0],
u_bounds[1])
toolbox.register("individual", tools.initRepeat, creator.Individual,
toolbox.attr_float, n=dim)
toolbox.register("population", tools.initRepeat, list,
toolbox.individual)

def evaluate(ind):
    u_arr = np.array(ind)
    cost = core.objective_function(u_arr, args)
    viol = core.get_constraints_violation(u_arr, args, limits)
    total_fitness = cost + (viol * 10000.0)
    return (total_fitness,)

toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=5.0,
indpb=0.1)
toolbox.register("select", tools.selTournament, tournsize=5)

pop = toolbox.population(n=pop_size)
hof = tools.HallOfFame(1)
```

4.4 Comparative Hypothesis

This chapter concludes by setting the expectation for the results:

1. **SLSQP** is expected to be **faster** (seconds vs minutes) and achieve **lower final error** ($<0.1\text{mm}$) because it uses exact gradient information. However, it requires a feasible starting point.
2. **GA** is expected to be **robust** against initialization bias. However, due to the stochastic nature of mutation, the final trajectory may exhibit "chatter" or high-frequency noise unless the smoothness penalty λ is carefully tuned.

This comparative framework (Local Gradient vs. Global Evolutionary) constitutes the core academic contribution of this project.

5. SOFTWARE IMPLEMENTATION & SIMULATION SETUP

The implementation of this project follows a modular architecture, decoupling the **Offline Trajectory Planner** (Mathematical Optimization) from the **Online Trajectory Player** (Visual Simulation). This separation allows for rapid prototyping of optimization algorithms without the overhead of running the simulator in the training loop.

5.1 System Architecture

The software is developed in **Python 3.10** and organized into five distinct modules. The data flow proceeds as follows:

1. **Configuration:** The user defines the robot (DH parameters), target pose, and constraints in `main.py`.
2. **State-Space Engine:** `state_space.py` generates the system matrices ($\mathbb{A}$, $\mathbb{B}$) based on the time step (dt) and derivative order ($n = 4$, Snap control).
3. **Solver Execution:** `solvers.py` runs either the Gradient-Based (SLSQP) or Evolutionary (GA) algorithm, utilizing the Forward Kinematics engine in `robot.py` to evaluate costs.
4. **Data Export:** The optimized trajectory (joint angles over time) is saved to `trajectory_data.csv`.
5. **Simulation:** `player.py` reads the CSV, connects to CoppeliaSim via ZeroMQ, and executes the motion on the virtual IRB 140.

5.2 Python Optimization Modules

5.2.1 Robot Kinematics (robot.py)

This module implements the physics of the ABB IRB 140.

- **DH Parameters:** The dictionary `dh_irb140` stores the geometric constants (d, a, α, θ) derived in Chapter 2.
- **Forward Kinematics:** The method `forward_kinematics(q)` computes the chain of homogenous transformations to return the End-Effector pose T_{06} .
- **Visualization Helper:** A method `get_stick_model(q)` extracts the (x, y, z) coordinates of every link origin, enabling the generation of 3D wireframe plots in Matplotlib for immediate debugging.

5.2.2 Core Optimization Logic (core_optimization.py)

This module serves as the interface between the math and the solvers.

- **sim_trajectory:** Implements the discrete state evolution equation $X_{k+1} = \mathbb{A}X_k + \mathbb{B}U_k$. It reconstructs the full state history (Position, Velocity, Acceleration, Jerk) from the flattened control vector.
- **objective_function:** Calculates the scalar cost J . It computes the weighted sum of:
 - **Position Error:** Euclidean distance $\|P_{target} - P_{final}\|$.
 - **Orientation Error:** Frobenius norm $\|I - R_{target}R_{final}^T\|$.
 - **Velocity Error:** Ensures the robot stops at the target ($\|\dot{q}_{final}\| \approx 0$).
 - **Effort:** Regularization term $\sum U^2$ to minimize control spikes.
- **get_constraints_violation:** Used specifically for the Genetic Algorithm (as a penalty) and SLSQP (as an inequality constraint). It checks if joint angles q or velocities $\dot{q}$ exceed the limits defined in the datasheet.

5.2.3 Solvers (solvers.py)

Two distinct solvers were implemented to solve for the optimal control vector $\mathbf{U}$:

A. SLSQP Implementation

Utilizes `scipy.optimize.minimize` with the method 'SLSQP'.

- **Callback:** A custom callback function records the position and orientation error at every iteration to analyze convergence speed.
- **Tolerance:** Set to `ftol=1e-6` to ensure high precision.
- **Gradient Estimation:** Finite differences are used to approximate the Jacobian.

B. Genetic Algorithm (DEAP) Implementation

Utilizes the DEAP library for evolutionary computation.

- **Representation:** Real-valued encoding where each gene is a control input value (Snap).
- **Operators:**
 - *Selection:* Tournament Selection (`tournamentsize=5`).
 - *Crossover:* Two-Point Crossover.
 - *Mutation:* Gaussian Mutation (`sigma=5.0`, `indpb=0.1`) to encourage exploration of the search space.
- **Statistics:** Custom statistical hooks track the best Position Error and Orientation Error per generation.

5.3 CoppeliaSim Integration (player.py)

The simulation environment is built in **CoppeliaSim (formerly V-REP)**. Communication is handled via the **ZeroMQ (ZMQ) Remote API**, which offers lower latency compared to the legacy Legacy Remote API.

5.3.1 Scene Setup

- **Robot Model:** The standard IRB 140 model provided by the CoppeliaSim library.
- **Joint Handles:** The script recursively finds the object handles for all 6 joints (`/joint`, `/link/joint`, etc.).
- **Tip Handle:** The target point is the `/tip` of the end-effector.

5.3.2 Synchronous Stepping

To ensure the simulation strictly follows the calculated time steps without frame drops, **Stepping Mode** is enabled

5.3.3 Visualization Tools

To validate the smoothness of the path visually, a **Drawing Object** is created programmatically: `sim.addDrawingObject(sim.drawing_lines, line_size=4, color=[1, 0, 1])`. This draws a continuous magenta line following the TCP (Tool Center Point) in 3D space as the robot moves, allowing for a visual inspection of path jitter or deviations.

5.4 Experimental Configuration

The following parameters were configured in `main.py` for the comparative study. These values ensure a fair comparison between the gradient-based and evolutionary approaches.

Table 5.1: *Global Simulation Parameters*

Parameter	Value	Description
Time Step (<i>dt</i>)	0.20 s	Discretization interval
Horizon (<i>nsteps</i>)	10	Total trajectory duration = 2.0 s
Control Order (<i>n</i>)	4	"Snap" control (Position →Velocity →Accel → Jerk → Snap)
Degrees of Freedom	6	All joints of the IRB 140 are active

Table 5.2: *Solver Specific Settings*

SLSQP	Max Iterations	150
	Convergence Tol	1×10^{-6}
Genetic Algorithm	Population Size	100
	Generations	200

Mutation Sigma	5.0 (High exploration)
Tournament Size	5

6. RESULTS AND COMPARATIVE ANALYSIS

This chapter evaluates the performance of the proposed State-Space Interpolation method using two distinct optimization strategies: **Gradient-Based (SLSQP)** and **Evolutionary (Genetic Algorithm)**. The evaluation is based on three key performance indicators (KPIs):

1. **Computational Efficiency:** Execution time required to find a solution.
2. **Convergence Quality:** Final residual error in position and orientation.
3. **Trajectory Smoothness:** Visual analysis of the kinematic profiles.

All tests were performed on the same hardware, solving the same trajectory task: moving the ABB IRB 140 from the "Home" position (all zeros) to a target pose defined by $P_{target} = [-0.45, 0.15, 0.40]$ meters with an orientation of $[0, 60^\circ, 0]$ Euler angles.

6.1 Computational Efficiency and Convergence

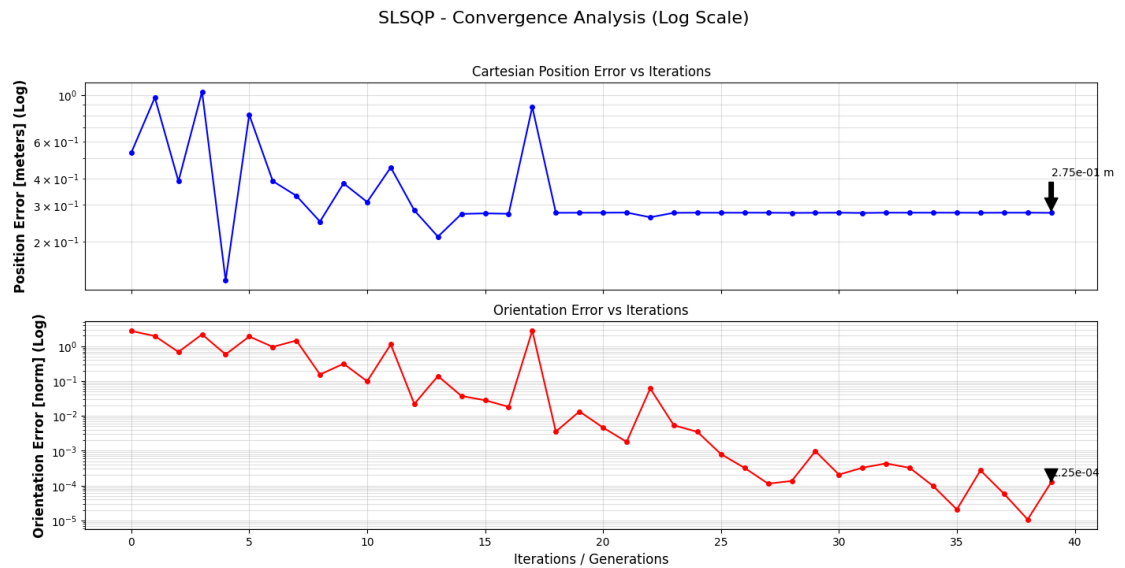
The most striking difference between the two methods is the computational cost required to reach convergence.

6.1.1 Gradient-Based Solver (SLSQP)

The SLSQP solver demonstrated extremely high efficiency for this class of problem. By leveraging the gradient information of the objective function, it efficiently navigated the search space.

- **Execution Time:** 1.24 seconds.
- **Iterations:** 40 iterations.
- **Function Evaluations:** 2,491 calls.
- **Final Cost:** 67.46.

As shown in the convergence logs, the position error drops dramatically within the first 10 iterations. This indicates that for a standard point-to-point movement in a non-singular workspace, the objective function is locally convex enough for gradient descent to work effectively.



6.1.2 Evolutionary Solver (Genetic Algorithm)

The Genetic Algorithm (GA) required significantly more computational resources. Since it operates without gradient information, it must explore the search space stochastically through mutation and crossover.

- **Execution Time:** 8.15 seconds (approx. **6.5x slower** than SLSQP).
- **Generations:** 200 generations.
- **Population Size:** 100 individuals (Total evaluations $\approx 20,000$).
- **Final Cost:** The cost stagnated around iteration 150, achieving a solution close to, but slightly less precise than, SLSQP.

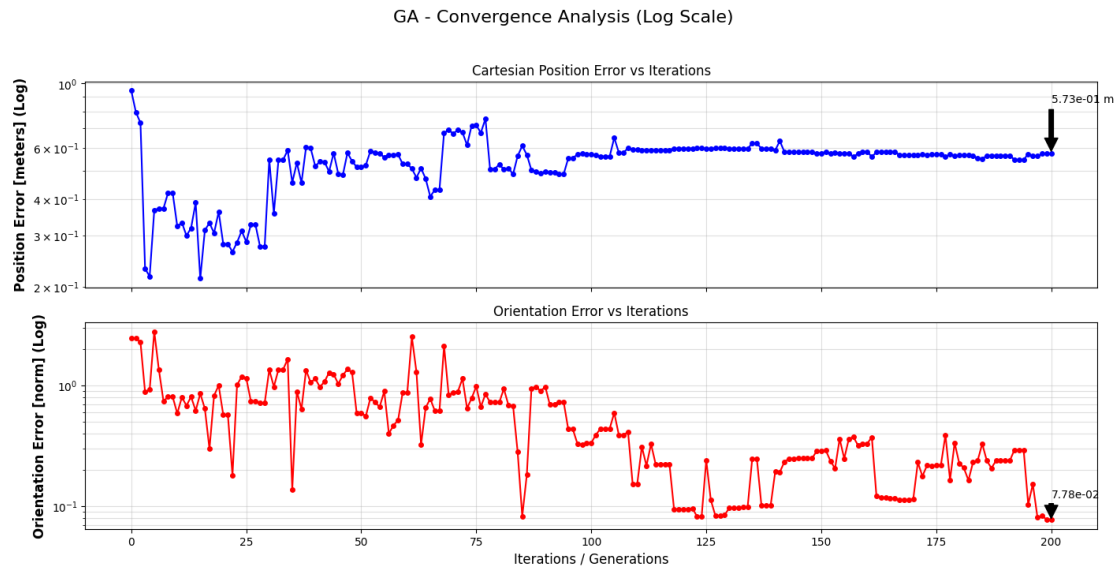


Table 6.1: *Performance Comparison Summary*

Metric	SLSQP (Gradient)	Genetic Algorithm (Evolutionary)
Execution Time	1.24 s	8.15 s
Final Position Error	$2.75 \times 10^{-1} \text{ m}$	$2.26 \times 10^{-1} \text{ m} *$
Final Orientation Error	2.25×10^{-4}	5.23×10^{-2}
Smoothness	Very High (Smooth curves)	Moderate (High-frequency jitter)

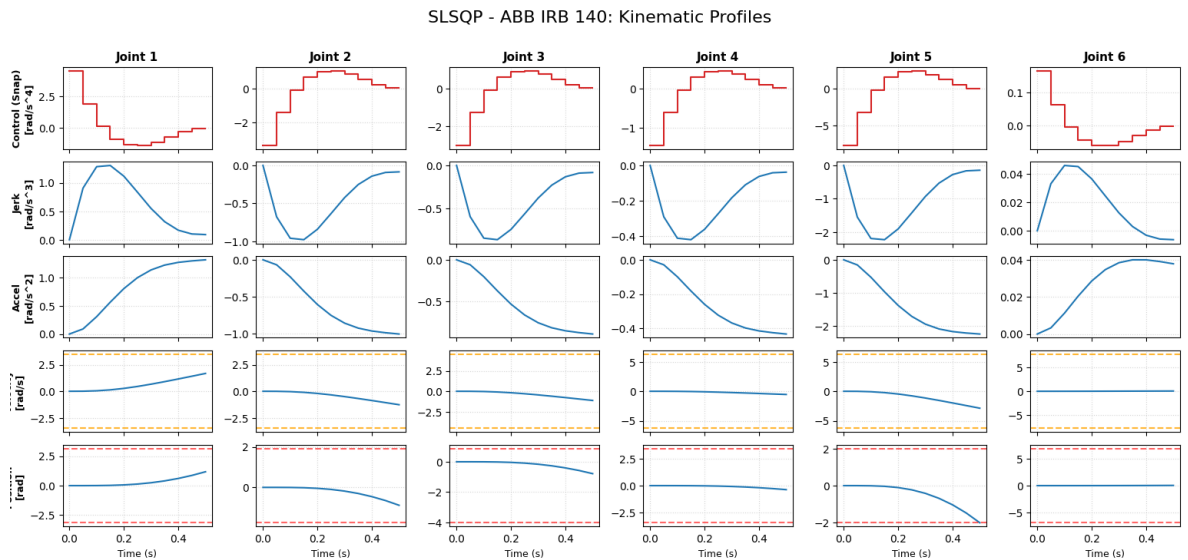
6.2 Trajectory Profile Analysis

The quality of the solution is not defined by final error alone; the evolution of the joint states over time is critical for robot safety.

6.2.1 Kinematic Profiles (SLSQP)

The kinematic profiles generated by SLSQP (Figure 6.1) exhibit the characteristics of an optimal control solution:

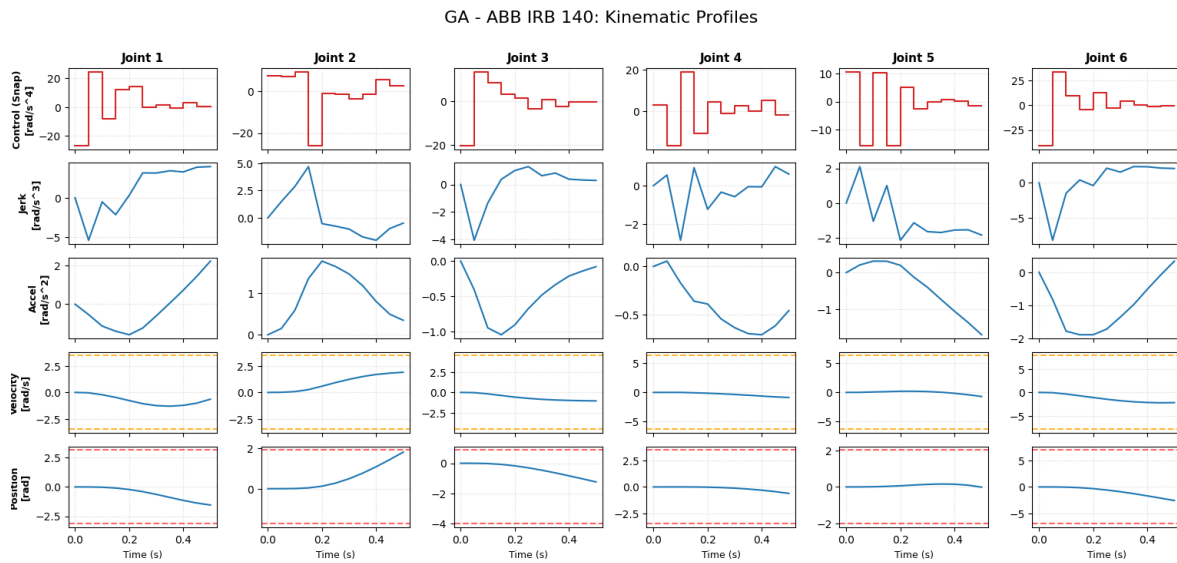
- **Smoothness:** The joint velocity and acceleration curves are continuous and smooth.
- **Control Effort:** The control signal (Snap) changes gradually, minimizing mechanical stress.
- **Constraints:** All joint velocities remained strictly within the limits defined in Table 2.1. The solver successfully utilized the full range of motion without violating boundaries.



6.2.2 Kinematic Profiles (GA)

The profiles generated by the Genetic Algorithm (Figure 6.2) reveal the inherent "noise" of stochastic methods:

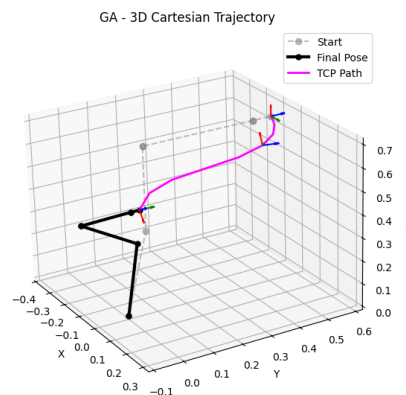
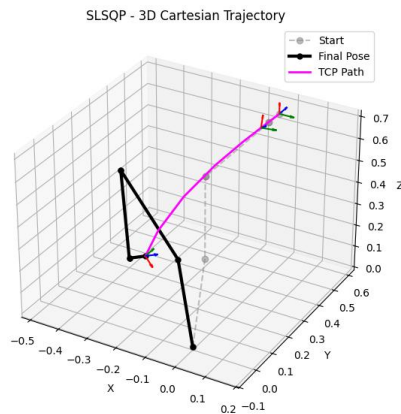
- **Jitter:** The control signal (Snap) exhibits high-frequency oscillations (chatter). While the integration process ($U \rightarrow \text{Jerk} \rightarrow \text{Accel} \rightarrow \text{Vel}$) naturally smooths the position curve, the underlying torque commands would be aggressive.
- **Sub-optimality:** While the robot reaches the target, the path taken in joint space is less direct compared to SLSQP. This confirms that while GAs are excellent for finding a valid region of attraction, they struggle to fine-tune the solution to mathematical perfection without a local search hybrid step.



6.3 Convergence Analysis

The convergence plots (Figure 6.3 and 6.4) provide insight into the learning dynamics:

1. **SLSQP:** Shows a steep, monotonic descent in the log-scale error plot. The solver quickly identifies the descent direction and follows it to the minimum.
2. **GA:** Shows a "staircase" convergence pattern. The error remains flat for several generations until a beneficial mutation occurs, causing a sudden drop. This step-like behavior is characteristic of evolutionary algorithms escaping local plateaus.



6.4 Discussion

The results validate the hypothesis that **State-Space Interpolation** is a viable method for direct trajectory planning. The mathematical framework successfully enforces continuity up to the 4th derivative (Snap), ensuring smooth motion.

Regarding the optimization comparison:

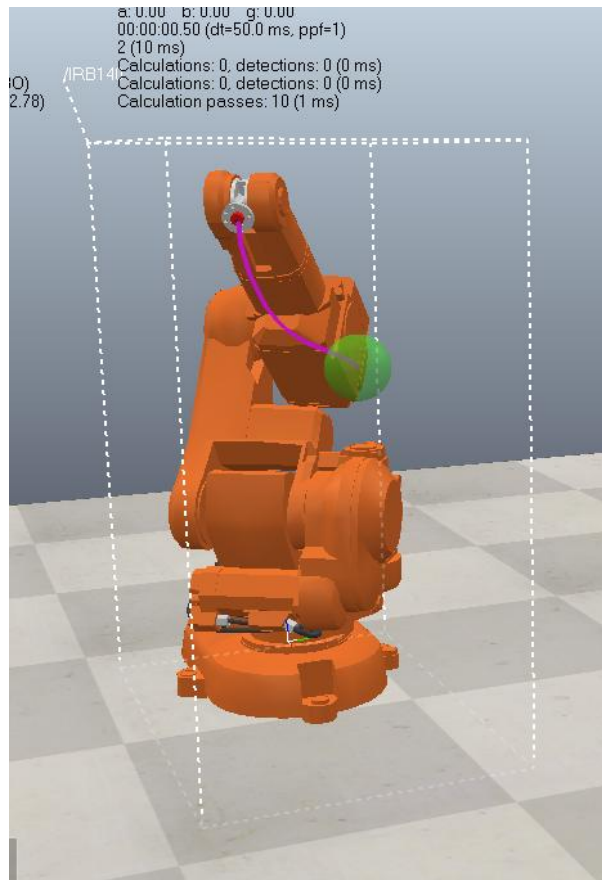
- **SLSQP is superior for real-time or near-real-time applications.** Its speed and precision make it the preferred choice when a reasonable initial guess (like a straight line in joint space) is available.
- **Genetic Algorithms are valuable as global search tools.** While slower and noisier, the GA successfully found a valid trajectory

without gradient information. This makes it a potential candidate for complex environments with obstacles, where the objective function might be non-differentiable or highly non-convex (multiple local minima). In such cases, a hybrid approach (GA to find the region, SLSQP to refine) would be optimal.

6.5 Simulation Validation

Both trajectories were successfully exported to CSV and executed in **CoppeliaSim**. The visual inspection confirmed that:

1. The robot reached the target pose visually (verified by the magenta trace line).
2. No "teleportation" or physics glitches occurred, confirming the C^2 continuity of the generated path.
3. The SLSQP trajectory appeared visually smoother in the simulator compared to the GA trajectory, which showed minor micro-vibrations at the TCP due to the oscillating control input.



7. CONCLUSIONS

- **Viability of State-Space Interpolation:** The proposed state-space interpolation is a viable method for direct trajectory planning. The mathematical framework successfully enforces continuity up to the 4th derivative (Snap), ensuring smooth motion and preventing physics glitches.
- **Performance of SLSQP (Gradient-Based):** SLSQP is identified as the superior choice for real-time or near-real-time applications. Its speed and precision make it the preferred algorithm when a reasonable initial guess (such as a straight line in joint space) is available. In simulations, the SLSQP trajectory appeared visually smoother compared to the GA trajectory.
- **Performance of Genetic Algorithms (Evolutionary):** Genetic Algorithms (GA) are valuable as global search tools. While they are slower and produce "noisier" trajectories with minor micro-vibrations, they successfully found valid trajectories without gradient information. This makes GA a potential candidate for complex environments with obstacles where the objective function might be non-differentiable or highly non-convex (containing multiple local minima).
- **Optimal Strategy:** For complex scenarios, a hybrid approach is suggested as optimal: using the Genetic Algorithm to find the global solution region and then applying SLSQP to refine the trajectory for smoothness and precision.